CLOUDSHIFT

Universal Code Refactoring Engine for Google Cloud Platform

Rust Core · Python Bindings via Maturin · Any Language → GCP

Product Requirements Document · Version 2.0 · March 2026

Author	Allan Smeyatsky / Smeyatsky Labs
Classification	**CONFIDENTIAL — Client Restricted**
Status	DRAFT v2.0
Target Release	Q3 2026 (MVP) · Q4 2026 (GA)

1. Executive Summary

CloudShift v2 is a universal code refactoring engine that transforms source code written in any programming language into idiomatic, production-ready Google Cloud Platform (GCP) code. Where v1 focused on infrastructure-level mapping (AWS/Azure → GCP service equivalents), v2 goes deeper: it understands application-layer code, SDK usage, runtime assumptions, and deployment targets, and rewrites them — in place — to be GCP-native.

🎯 Core Mission: Any developer, on any codebase, in any language, should be able to point CloudShift at their code and receive GCP-ready output in minutes — not weeks. No cloud expertise required. No manual pattern lookup. No copy-paste from documentation.

The engine is built on three immovable technical decisions:

- **Rust Core:** The transformation engine, AST parser bridge, pattern compiler, and diff emitter are written in Rust. This delivers deterministic, memory-safe, sub-second transformation times even on large repositories.
- **Python Bindings via Maturin:** The Rust core is exposed as a native Python extension module (PyO3 + Maturin). This makes CloudShift importable as cloudshift in any Python environment — usable from notebooks, CI pipelines, data engineering workflows, and ML platforms without any additional runtime.
- **Tree-sitter for AST Parsing:** Tree-sitter grammars provide incremental, error-tolerant parsing for 40+ languages. CloudShift operates on the AST — not on regex or text — making transformations structurally correct by construction.

CloudShift v2 competes directly with the AI-assisted refactoring tools identified in the market (Augment Code, GitHub Copilot, Cline, Aider) by offering something none of them provide: a GCP-specialised, compiled transformation engine that is deterministic for known patterns, fast enough to run in a CI gate, and auditable for compliance purposes.

🏆 Differentiator vs. General-Purpose AI Refactoring Tools: CloudShift is not a general-purpose coding assistant. It is a directed transformation engine with a single, well-defined target: GCP. This specificity is its moat. Every pattern it ships is validated against real GCP services. Every output is a valid, deployable GCP artefact.

2. Problem Statement

2.1 The GCP Adoption Gap

Google Cloud Platform is the fastest-growing hyperscaler in enterprise AI/ML, analytics, and data workloads. Despite this growth, teams that want to migrate existing applications to GCP face a consistent, painful bottleneck: their existing code is written for other clouds, other runtimes, or other deployment models, and there is no tool that reliably, automatically translates it.

The industry benchmark from the Augment Code research cited by practitioners shows legacy code refactoring consumes 40% of engineering time. For cloud migration specifically, the figure is higher: 60-70% of migration effort is consumed by code-level changes that could, in principle, be automated.

2.2 Why Existing Tools Fall Short for GCP Migration

Tool Category	What They Do Well	GCP Migration Gap
Static Linters (ESLint, SonarQube)	Catch code smells, enforce style	No awareness of cloud SDK semantics or GCP service mappings
AI Assistants (Copilot, Cursor)	Suggest rewrites interactively	Non-deterministic, requires expert review, no GCP specialisation
Context-Aware AI (Augment Code)	Repo-wide refactoring with large context	General purpose — no compiled GCP mapping, no CLI/CI integration for migration
Cloud Migration Tools (GCP Migrate)	VM/container lift-and-shift	Does not touch application code, IaC, or SDK calls
Manual Architect Work	Accurate when done well	Expensive, slow, knowledge locked in individuals, not scalable

2.3 The Root Cause

The root cause is that no existing tool treats GCP migration as a first-class, compiled transformation problem. It is treated as a documentation-lookup problem (developers read the GCP docs and rewrite manually) or as a general AI completion problem (ask a model to rewrite). Neither approach scales, and neither produces auditable, consistently correct output.

CloudShift v2 solves this by treating GCP migration as what it actually is: a deterministic mapping problem with a well-defined source space (all cloud SDK patterns) and a well-defined target space (GCP equivalents). That problem is solvable with a compiled engine. The cases where it is not deterministic (novel patterns, business logic intertwined with infrastructure) are handled by an AI-assisted fallback — but those are the exception, not the rule.

2.4 The Language Universality Problem

Existing GCP migration accelerators, where they exist at all, target a single language or framework. CloudShift v2 is built from the ground up as language-agnostic. The transformation pipeline operates on Tree-sitter ASTs, which support 40+ languages through a unified interface. The GCP pattern catalogue is language-aware but not language-specific — the same logical mapping (e.g., 'replace S3 put_object with GCS blob.upload_from_string') is compiled to correct transformations in Python, JavaScript, Java, Go, and C# separately but from a single source-of-truth mapping definition.

3. Goals and Non-Goals

3.1 Goals — v2.0 MVP (Q3 2026)

1. Transform application code using AWS SDK (boto3, AWS SDK for Java, aws-sdk-js, etc.) to GCP SDK equivalents across Python, TypeScript/JavaScript, Java, and Go.
2. Transform application code using Azure SDK (azure-storage-blob, Azure.Storage, @azure/storage-blob, etc.) to GCP SDK equivalents across the same language set.
3. Transform Infrastructure as Code: Terraform HCL (AWS provider → Google provider), CloudFormation YAML → Terraform Google, Bicep → Terraform Google.
4. Transform Dockerfile and container build artefacts to GCP Cloud Run and Artifact Registry conventions.
5. Transform CI/CD pipeline definitions: GitHub Actions, GitLab CI, Bitbucket Pipelines → Cloud Build / Cloud Deploy YAML.
6. Transform environment variable patterns and secrets management (AWS Secrets Manager, Azure Key Vault → Secret Manager).
7. Handle monorepo and multi-file transformations with a dependency graph traversal — not just single-file snippets.
8. Ship a Rust core library, a Python extension module (via Maturin + PyO3), and a CLI binary (cloudshift).
9. Produce a diff-first output: every transformation is a reviewable, apply-able diff before any file is mutated.
10. Maintain a GCP Pattern Catalogue with 200+ compiled rules at MVP, extensible via a TOML/YAML rule DSL.

3.2 Goals — GA (Q4 2026)

11. Expand language coverage to include C#, Ruby, Kotlin, Scala, and Rust itself.
12. Add AI-assisted fallback via a local LLM (Ollama) for low-confidence and novel patterns.
13. Add a self-learning pattern store: approved AI-generated transformations are promoted to compiled rules.
14. Ship VS Code and JetBrains IDE extensions powered by the same Rust core via LSP.
15. Add a Validation Agent: post-transformation checks including Terraform plan diff, test re-execution, and GCP emulator smoke tests.
16. Support Pulumi (TypeScript, Python, Go) as an IaC source.

3.3 Non-Goals

- Data migration — CloudShift transforms code that talks to data, not the data itself.
- Network topology migration — VPC, firewall rules, peering are out of scope.
- VM or container lift-and-shift — GCP Migrate for Compute Engine handles this.
- Performance optimisation beyond service substitution — CloudShift produces equivalent code, not optimised code.
- Multi-target output — the target is always GCP. CloudShift is not a multi-cloud abstraction layer.
- General-purpose refactoring (code quality, style, architecture) — CloudShift does one thing: GCP migration.

4. Technical Architecture

4.1 Architecture Philosophy

🏗 Architecture Principle: The transformation engine must be deterministic, testable, and fast. Every design decision is subordinate to this. Rust is chosen because it is the only systems language that delivers memory safety, zero-cost abstractions, and a rich ecosystem for compiler-adjacent work (syn, proc-macro, tree-sitter bindings) without a garbage collector pause or an interpreter overhead.

4.2 Component Diagram

CloudShift v2 is composed of six major components, all within the cloudshift-core Rust crate:

Component	Language	Responsibility
Ingestion Layer	Rust	Walks file trees, detects languages, queues files for analysis. Handles Git repos, directories, single files, and stdin.
Tree-sitter Bridge	Rust (tree-sitter crate)	Parses source files into language-specific CSTs. Maintains incremental parse trees for large repos.
Semantic Analyser	Rust	Traverses the CST to extract cloud-relevant constructs: SDK imports, function calls, env vars, resource definitions, IAM references.
Pattern Engine	Rust	Loads compiled patterns from the GCP Pattern Catalogue. Matches semantic constructs against patterns. Emits structured AST diffs with confidence scores.
Diff Emitter	Rust	Converts AST diffs to unified diffs, language-native patch formats, or direct file mutations. Never mutates without explicit apply.
Python Extension (PyO3)	Rust + Python (Maturin)	Exposes the full transformation pipeline as a native Python module. Zero FFI overhead. Installable via pip.

4.3 Rust Core — Crate Structure

The Rust workspace is structured as a clean set of crates with clear dependency boundaries:

```
cloudshift/
├── crates/
│   ├── cloudshift-core/      # Pure transformation engine, no I/O
│   │   ├── src/
│   │   │   ├── analyser/     # Language-specific semantic extractors
│   │   │   ├── pattern/      # Pattern DSL compiler & matcher
│   │   │   ├── diff/         # AST diff → unified diff emitter
│   │   │   └── catalogue/    # GCP pattern catalogue loader
│   │   └── Cargo.toml
│   ├── cloudshift-cli/       # CLI binary (clap + tokio)
│   │   └── Cargo.toml
│   ├── cloudshift-lsp/       # LSP server for IDE extensions
│   │   └── Cargo.toml
```

```
│   └── cloudshift-py/        # PyO3 bindings (built by Maturin)
│       ├── src/lib.rs
│       └── Cargo.toml
├── patterns/                 # GCP Pattern Catalogue (TOML)
│   ├── python/
│   ├── typescript/
│   ├── java/
│   ├── go/
│   └── hcl/
├── Cargo.toml                # Workspace root
└── pyproject.toml            # Maturin build config
```

4.4 Python Extension via Maturin + PyO3

Maturin is the build backend that compiles the cloudshift-py Rust crate into a native Python extension wheel. This is the standard approach for shipping high-performance Rust code as a Python package, used by production libraries including Polars, Pydantic v2, and cryptography.

> 🧠 Why Maturin over CFFI or ctypes: Maturin produces a proper PEP 517 wheel with platform-native binaries. Users install it with pip install cloudshift — no Rust toolchain required on the user's machine. PyO3 provides safe, ergonomic Rust↔Python bindings with automatic type conversion for strings, lists, dicts, and bytes.

The Python API surface is intentionally minimal and Pythonic:

```
import cloudshift

# Transform a single file
result = cloudshift.transform_file(
    path='src/storage.py',
    source_cloud='aws',
    target='gcp',
)
print(result.diff)          # unified diff string
print(result.confidence)    # float 0.0-1.0
result.apply()              # mutates file in place

# Transform an entire repository
report = cloudshift.transform_repo(
    path='./my-app',
    source_cloud='aws',   # or 'azure', or 'any'
    dry_run=True,         # default: always dry-run first
)
for change in report.changes:
    print(f'{change.file}: {change.patterns_matched} patterns, ',
          f'confidence={change.confidence:.2f}')
```

4.5 Tree-sitter Integration

Tree-sitter provides incremental, error-tolerant parsing for every language CloudShift supports. The tree-sitter Rust crate is the foundation; language-specific grammars are compiled as Rust static libraries and linked directly into cloudshift-core. This means:

- No subprocess spawning, no grammar downloads at runtime — all parsers are compiled in.
- Error-tolerant parsing: real-world code with syntax errors is parsed as best-effort, not rejected.
- Incremental re-parsing: on large repos, only changed files are re-parsed.
- Unified query interface: Tree-sitter S-expression queries are used for pattern matching across all languages.

Language grammar inclusion at MVP:

```
# Cargo.toml (cloudshift-core)
[dependencies]
tree-sitter = "0.22"
tree-sitter-python = "0.21"
tree-sitter-javascript = "0.21"
tree-sitter-typescript = "0.21"
tree-sitter-java = "0.21"
tree-sitter-go = "0.21"
tree-sitter-hcl = "0.1"       # Terraform HCL
tree-sitter-yaml = "0.6"       # CloudFormation, CI/CD YAML
tree-sitter-dockerfile = "0.2"
tree-sitter-json = "0.21"
```

4.6 The GCP Pattern Catalogue

The Pattern Catalogue is the heart of CloudShift's deterministic capability. It is a structured collection of source-to-target transformation rules, written in a simple TOML-based DSL and compiled into the Rust binary at build time (no runtime loading required, though hot-reload is supported for development).

Pattern DSL Structure

```
# patterns/python/aws_s3_to_gcs.toml

[pattern]
id         = "aws.s3.put_object -> gcp.gcs.blob.upload"
source     = "aws"
language   = "python"
confidence = 0.97
tags       = ["storage", "s3", "gcs"]

[pattern.detect]
# Tree-sitter S-expression query to identify the source pattern
query = '''
  (call
    function: (attribute
      object: (identifier) @client
      attribute: (identifier) @method (#eq? @method "put_object"))
    arguments: (argument_list) @args)
'''
imports = ["boto3"]
```

```
[pattern.transform]
# Template for the replacement AST node
template = '''
  {bucket_var}.blob({key_expr}).upload_from_string({body_expr})
'''
import_add    = ["from google.cloud import storage"]
import_remove = ["import boto3"]

[pattern.bindings]
# How to extract template variables from the detected AST
bucket_var = "args.Bucket"
key_expr   = "args.Key"
body_expr  = "args.Body"
```

4.7 Confidence Scoring Model

Every pattern application produces a confidence score between 0.0 and 1.0. The score is calculated from four factors:

Factor	Weight	Description
Pattern specificity	35%	How precisely the pattern matches the detected construct. Exact import + method match = high specificity.
Version alignment	25%	Whether the detected SDK version matches the pattern's validated version range.
Argument completeness	25%	Percentage of required template variables successfully bound from the detected AST.
Test coverage	15%	Whether the pattern has associated test fixtures that passed in CI.

Thresholds:

- **≥ 0.90:** Auto-transform in --auto mode. Shown as 'High Confidence' in interactive mode.
- **0.70 – 0.89:** Transform with review flag. Diff is generated but apply requires explicit confirmation.
- **< 0.70:** Flagged for AI-assisted fallback (GA) or manual review (MVP). Diff is still generated as best-effort.

5. GCP Target Mapping — Service Coverage

The following table defines the service mapping coverage for CloudShift v2 MVP. Each mapping represents a compiled pattern set covering the source SDK calls in at least Python and TypeScript at MVP.

5.1 Compute & Serverless

Source (AWS)	Source (Azure)	GCP Target	Transform Type	MVP
Lambda (boto3)	Azure Functions	Cloud Functions (Gen 2) / Cloud Run	Code + IaC	Yes
ECS / Fargate	Azure Container Apps	Cloud Run	Dockerfile + IaC	Yes
EC2 (via SDK)	Azure VMs (via SDK)	Compute Engine	IaC only	Partial
Elastic Beanstalk	Azure App Service	Cloud Run + Cloud Build	IaC + CI	Yes
Step Functions	Logic Apps / Durable Functions	Cloud Workflows	IaC + YAML	No (GA)

5.2 Storage

Source (AWS)	Source (Azure)	GCP Target	SDK Transform	MVP
S3 (boto3, sdk-js, sdk-java)	Azure Blob Storage	Cloud Storage (GCS)	Yes	Yes
EFS / FSx	Azure Files	Filestore	IaC only	No (GA)
Glacier	Azure Archive	GCS Archive class	IaC only	Yes
S3 Presigned URLs	SAS Tokens	GCS Signed URLs	Code transform	Yes

5.3 Databases & Analytics

Source (AWS)	Source (Azure)	GCP Target	Transform Type	MVP
DynamoDB (boto3)	Cosmos DB	Firestore / Bigtable	Code + IaC	Yes
RDS (psycopg2, JDBC)	Azure SQL / Postgres	Cloud SQL / AlloyDB	Connection string + IaC	Yes
Aurora	Azure SQL Managed Instance	AlloyDB / Spanner	IaC + code hints	Partial
Athena (boto3)	Azure Synapse	BigQuery	Code + SQL dialect	Yes
Kinesis Data Streams	Event Hubs	Pub/Sub	Code + IaC	Yes
Kinesis Firehose	Event Hubs Capture	Dataflow + GCS	IaC + code	No (GA)

Source (AWS)	Source (Azure)	GCP Target	Transform Type	MVP
Redshift	Azure Synapse Analytics	BigQuery	IaC + SQL	Yes

5.4 Auth, Security & Identity

Source (AWS)	Source (Azure)	GCP Target	Transform Type	MVP
IAM roles/policies (Terraform)	Azure RBAC (Bicep/ARM)	IAM (Terraform Google)	IaC transform	Yes
Secrets Manager	Azure Key Vault	Secret Manager	Code + IaC	Yes
Cognito	Azure AD B2C	Identity Platform / Firebase Auth	IaC + code hints	No (GA)
STS AssumeRole	Managed Identity	Workload Identity Federation	Code + IaC	Yes

5.5 AI/ML & Data

Source (AWS)	Source (Azure)	GCP Target	Transform Type	MVP
SageMaker (boto3)	Azure ML SDK	Vertex AI	Code + IaC	Yes
Bedrock (boto3)	Azure OpenAI	Vertex AI Generative AI	Code transform	Yes
Rekognition	Azure Computer Vision	Vision AI	Code transform	Yes
Comprehend	Azure Text Analytics	Natural Language AI	Code transform	Yes

6. CLI Specification

6.1 Core Commands

```
cloudshift [COMMAND] [OPTIONS]

Commands:
  transform    Transform code files or repositories
  analyse      Analyse code without transforming (detect patterns only)
  diff         Show what would change without applying
  apply        Apply a previously generated diff
  catalogue    Manage and query the GCP Pattern Catalogue
  validate     Run post-transformation validation checks
  report       Generate a migration report from a previous run
```

6.2 Transform Command

```
cloudshift transform [PATH] [OPTIONS]

Arguments:
  PATH                File, directory, or Git repo URL (default: .)

Options:
  --source <CLOUD>     Source cloud: aws | azure | any  [default: any]
  --language <LANG>    Restrict to language: python | typescript | java | go | hcl
| all
  --dry-run            Show diff, do not apply  [default: true]
  --auto               Auto-apply high-confidence transformations (≥0.90)
  --threshold <FLOAT>  Minimum confidence to include in diff  [default: 0.70]
  --output <FORMAT>    Output format: diff | json | sarif  [default: diff]
  --report <PATH>      Write JSON migration report to path
  --parallel <N>       Worker threads for repo-level transforms  [default: ncpu]
  --no-iac             Skip Infrastructure as Code files
  --no-ci              Skip CI/CD pipeline files
  --include <GLOB>     Include files matching glob pattern
  --exclude <GLOB>     Exclude files matching glob pattern

Examples:
  cloudshift transform ./my-app --source aws --dry-run
  cloudshift transform src/storage.py --auto
  cloudshift transform . --output json --report migration-report.json
```

6.3 Analyse Command

```
cloudshift analyse [PATH] [OPTIONS]

# Produces a Migration Manifest: all detected cloud constructs,
# matched patterns, confidence scores, and estimated effort.

cloudshift analyse ./my-app --output json > manifest.json
```

```
cloudshift analyse ./my-app --format table   # human-readable summary

# Example output (table format):
#
# File                    Constructs  Patterns  Confidence  Effort
# src/storage.py          12          11         0.94        Low
# src/auth.py             8           7          0.88        Low
# src/analytics/etl.py    24          18         0.76        Medium
# infra/main.tf           45          44         0.96        Low
# .github/workflows/ci.yml 6           5          0.91        Low
# ──────────────────────────────────────────────────────────────────
# TOTAL                   95          85 (89%)   0.91 avg    Low-Medium
```

7. Python SDK (cloudshift)

7.1 Installation

```
pip install cloudshift

# The wheel is a native extension compiled for your platform.
# No Rust toolchain required. Supported platforms:
#   - Linux x86_64 (manylinux2014)
#   - Linux aarch64 (manylinux2014)
#   - macOS x86_64 (10.12+)
#   - macOS aarch64 (Apple Silicon)
#   - Windows x86_64
```

7.2 Core API

```
import cloudshift
from cloudshift import TransformConfig, SourceCloud, OutputFormat

# ── Single file transform ──────────────────────────────────────
result = cloudshift.transform_file(
    path='src/storage.py',
    config=TransformConfig(
        source_cloud=SourceCloud.AWS,
        threshold=0.85,
        dry_run=True,
    )
)

# result.diff          -> str (unified diff)
# result.patterns      -> list[PatternMatch]
# result.confidence    -> float
# result.warnings      -> list[Warning]
result.apply()  # applies the diff in place

# ── Repository transform ───────────────────────────────────────
report = cloudshift.transform_repo(
    path='./my-app',
    config=TransformConfig(
        source_cloud=SourceCloud.ANY,
        auto_apply_threshold=0.90,
    )
)

# ── Streaming transform (large repos) ──────────────────────────
for file_result in cloudshift.transform_repo_stream('./my-app'):
    print(f'{file_result.path}: {file_result.confidence:.2f}')

# ── Catalogue queries ──────────────────────────────────────────
patterns = cloudshift.catalogue.search('s3')
for p in patterns:
    print(p.id, p.language, p.confidence)
```

7.3 Jupyter / Notebook Integration

CloudShift ships first-class Jupyter integration. In a notebook context, result.diff renders as a syntax-highlighted HTML diff block, and result.apply() is guarded by an interactive confirmation widget.

```
# In a Jupyter cell:
import cloudshift.notebook as cs_nb

cs_nb.analyse('./my-app')  # renders a visual migration manifest
cs_nb.diff('./my-app')     # renders a syntax-highlighted diff explorer
```

7.4 CI/CD Integration

```
# .github/workflows/cloudshift.yml
name: CloudShift Migration Gate
on: [pull_request]

jobs:
  cloudshift:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Install CloudShift
        run: pip install cloudshift

      - name: Run analysis
        run: |
          cloudshift analyse . --output json > manifest.json
          cloudshift transform . --dry-run --output sarif > results.sarif

      - name: Upload SARIF
        uses: github/codeql-action/upload-sarif@v3
        with:
          sarif_file: results.sarif
```

8. Maturin Build Architecture

8.1 pyproject.toml Configuration

```
[build-system]
requires      = ["maturin>=1.5,<2"]
build-backend = "maturin"

[project]
name        = "cloudshift"
description = "Universal GCP code refactoring engine"
version     = "2.0.0"
requires-python = ">=3.9"
classifiers = [
    "Programming Language :: Python :: 3",
    "Programming Language :: Rust",
    "Topic :: Software Development :: Code Generators",
]

[tool.maturin]
features        = ["pyo3/extension-module"]
python-source   = "python"
module-name     = "cloudshift._cloudshift_core"
manifest-path   = "crates/cloudshift-py/Cargo.toml"
strip           = true
profile         = "release"
```

8.2 PyO3 Binding Layer

```
// crates/cloudshift-py/src/lib.rs
use pyo3::prelude::*;
use cloudshift_core::{transform_file, transform_repo, TransformConfig};

#[pyfunction]
fn transform_file_py(
    py: Python<'_>,
    path: &str,
    source_cloud: Option<&str>,
    threshold: Option<f64>,
    dry_run: Option<bool>,
) -> PyResult<PyTransformResult> {
    let config = TransformConfig {
        source_cloud: source_cloud.unwrap_or("any").parse()?,
        threshold:    threshold.unwrap_or(0.70),
        dry_run:      dry_run.unwrap_or(true),
    };
    let result = py.allow_threads(|| transform_file(path, config))?;
    Ok(result.into())
}

#[pymodule]
fn _cloudshift_core(m: &Bound<'_, PyModule>) -> PyResult<()> {
    m.add_function(wrap_pyfunction!(transform_file_py, m)?)?;
```

```
    m.add_function(wrap_pyfunction!(transform_repo_py, m)?)?;
    Ok(())
}
```

🥷 Build Pipeline: maturin build --release produces platform-specific wheels. The CI pipeline builds wheels for all supported platforms using GitHub Actions with cross-compilation for Linux ARM via cross. All wheels are published to PyPI. The CLI binary is published as a GitHub Release asset and via Homebrew tap (smeyatsky-labs/tap).

8.3 Release Matrix

Platform	Architecture	Build Method	Distribution
Linux	x86_64	maturin build --release (manylinux2014)	PyPI wheel + GitHub Release
Linux	aarch64	cross compile via cross	PyPI wheel + GitHub Release
macOS	x86_64	maturin build --release	PyPI wheel + Homebrew
macOS	aarch64 (Apple Silicon)	maturin build --release	PyPI wheel + Homebrew
Windows	x86_64	maturin build --release (Windows runner)	PyPI wheel + MSI installer

9. User Personas and Experience

9.1 Primary Personas

Persona	Role	Primary Need	Primary Surface
The Migrator	Cloud/Platform Engineer	Bulk-transform a repository with high confidence and an audit trail	CLI / Python SDK
The Developer	Full-Stack / Backend Developer	Refactor a single file or function mid-sprint without learning GCP APIs	IDE Extension / CLI
The Data Engineer	Data / ML Engineer	Transform data pipeline code (boto3/Azure SDK) to BigQuery/Vertex AI	Python SDK / Notebook
The Architect	Solutions / Enterprise Architect	Review a migration plan, assess effort, audit transformation decisions	CLI report output / Web UI (GA)
The DevOps Lead	Platform / DevOps Engineer	Gate pull requests against GCP migration completeness in CI	GitHub Actions / CLI
The Compliance Officer	CISO / GRC	Confirm no code egress and access an immutable transformation audit log	CLI report output

9.2 Key User Journeys

Journey 1 — The Migrator: Full Repository Transform

17. Run cloudshift analyse . to get the Migration Manifest. Review service coverage, confidence, and estimated effort.
18. Run cloudshift diff . > migration.patch to see the full unified diff without touching any files.
19. Review the diff in a PR (cloudshift supports GitHub diff preview via SARIF output).
20. Run cloudshift transform . --auto to apply all high-confidence transformations automatically.
21. Run cloudshift transform . to interactively review and apply medium-confidence transformations.
22. Run cloudshift validate . (GA) to confirm semantic equivalence.
23. Commit the changes. The migration report is part of the commit artefacts.

Journey 2 — The Data Engineer: SDK Swap in a Notebook

24. Open a Jupyter notebook containing boto3 S3 and SageMaker calls.
25. import cloudshift.notebook as cs; cs.analyse('.') renders a visual manifest.
26. cs.diff('.') renders a diff explorer in the notebook.
27. cs.apply('.') with a confirmation widget applies the changes.
28. The notebook now uses google.cloud.storage and vertexai.

Journey 3 — The DevOps Lead: CI Gate

29. Add cloudshift YAML step to the repository's CI pipeline.
30. Every PR is analysed. The SARIF output is uploaded and displayed as code scanning alerts.

31. PRs with AWS/Azure SDK calls that have high-confidence GCP equivalents are flagged for migration.
32. The migration gate can be set to warn-only or blocking based on team policy.

10. GCP Pattern Catalogue — Governance

10.1 Catalogue Design Principles

- Single source of truth: patterns are defined once in TOML and compiled to all target languages.
- Test-first: every pattern ships with at least one before/after fixture test. No untested pattern enters the catalogue.
- Versioned: patterns carry a source SDK version range and a GCP SDK version range. Version mismatches produce a warning.
- Community-extensible: the catalogue is open-source. Community contributions follow a review process with automated testing.
- Immutable history: pattern changes are tracked in Git. Every applied transformation records the pattern ID and version.

10.2 MVP Catalogue Size Targets

Category	Python	TypeScript	Java	Go	HCL (Terraform)	Total MVP
Compute / Serverless	18	12	8	8	15	61
Storage	22	14	10	8	12	66
Databases	20	10	8	6	14	58
Messaging / Events	14	10	6	6	10	46
Auth / IAM	12	8	6	4	12	42
AI / ML	16	8	6	4	8	42
CI/CD & Dockerfile	—	—	—	—	20	20
TOTAL	102	62	44	36	91	335

📊 335 compiled patterns at MVP positions CloudShift ahead of every existing open-source GCP migration tool and at parity with the most mature commercial offerings — but with the advantage of being deterministic, testable, and auditable rather than AI-generated.

10.3 Pattern Testing Framework

Every pattern in the catalogue has a corresponding test fixture in the tests/patterns/ directory. The test structure is consistent:

```
tests/patterns/python/aws_s3_to_gcs/
├── before.py       # Source code with AWS pattern
├── after.py        # Expected GCP output
└── meta.toml       # Pattern ID, confidence, expected warnings

# Test execution (via cargo test):
# 1. Parse before.py with Tree-sitter
# 2. Run Pattern Engine
# 3. Apply diff
# 4. Assert output == after.py (normalised)
```

```
# 5. Assert confidence >= meta.toml::expected_confidence
```

11. AI-Assisted Fallback (GA Feature)

11.1 Design Principle

⚠ The AI fallback is the last resort, not the first tool. CloudShift's compiled pattern engine handles 90%+ of transformations without AI. The fallback exists only for patterns with confidence < 0.70 that the catalogue does not yet cover. Crucially, every AI-generated transformation is a candidate for promotion to a compiled pattern — the fallback is a pattern discovery mechanism, not a permanent dependency.

11.2 Fallback Architecture

When a detected construct falls below the confidence threshold and no pattern matches:

33. The Semantic Analyser extracts a minimal AST context — the affected node, its parent context, and its imports. No full file content is sent.
34. The context is formatted as a structured prompt with explicit GCP target instructions.
35. The prompt is sent to the configured LLM endpoint (Ollama local, vLLM self-hosted, or client-owned cloud endpoint).
36. The LLM response is parsed for a code transformation suggestion.
37. The suggestion is wrapped as a low-confidence diff and presented for human review.
38. When a human approves and applies the transformation, the source/target AST pair is extracted and stored in the local Pattern Store.
39. After N approved uses (default: 5), the pattern is auto-promoted to the compiled catalogue.

11.3 LLM Configuration

```
# cloudshift.toml (project-level configuration)
[ai]
enabled  = false   # MVP default: AI fallback disabled
provider = "ollama"

[ai.ollama]
base_url = "http://localhost:11434"
model    = "codellama:13b"

# Alternative: self-hosted vLLM
[ai.vllm]
base_url = "https://llm.internal.corp/v1"
model    = "deepseek-coder-33b"
api_key  = "${VLLM_API_KEY}"

# Alternative: client-owned Vertex AI endpoint
[ai.vertex]
project  = "my-gcp-project"
location = "us-central1"
model    = "code-bison@002"
```

12. Development Roadmap

12.1 Phase 1 — Core Engine (Weeks 1-4)

- **Week 1-2:** cloudshift-core crate skeleton. Tree-sitter integration for Python and TypeScript. Ingestion layer (file walker, language detector). Basic CLI binary (cloudshift analyse).
- **Week 3-4:** Pattern DSL compiler (TOML → compiled Rust structs). First 50 Python patterns (S3, DynamoDB, Lambda, IAM). Test framework and fixture runner. cloudshift diff command.

12.2 Phase 2 — Python SDK and CI Integration (Weeks 5-8)

- **Week 5-6:** PyO3 binding layer. Maturin build pipeline. PyPI publishing. pip install cloudshift functional.
- **Week 7-8:** TypeScript/JavaScript patterns (50). Java patterns (44). HCL/Terraform patterns (50). cloudshift transform --auto command. SARIF output format. GitHub Actions example workflow.

12.3 Phase 3 — MVP Release (Weeks 9-12)

- **Week 9-10:** Go patterns (36). IaC transforms (CloudFormation → Terraform Google, Bicep → Terraform Google). Dockerfile → Cloud Run transforms. Remaining 91 HCL patterns.
- **Week 11-12:** End-to-end test suite on 3 real-world open-source applications. Confidence model calibration. Docs site (cloudshift.dev). MVP release: CLI binary, PyPI wheel, GitHub Actions.

12.4 Phase 4 — GA (Weeks 13-20)

- **Week 13-15:** AI fallback layer (Ollama integration, pattern store, auto-promotion). Extended language coverage: C#, Ruby, Kotlin.
- **Week 16-18:** LSP server (cloudshift-lsp). VS Code extension. JetBrains plugin. Validation Agent (Terraform plan diff, test re-execution).
- **Week 19-20:** Pulumi support. Web UI (Tauri desktop app wrapping the Rust core). GA release.

12.5 Milestone Summary

Milestone	Date	Deliverable	Success Metric
Alpha	Week 4	Core engine, Python patterns, CLI	50 Python patterns, 95%+ fixture pass rate
Beta	Week 8	Python SDK, TS/Java/HCL patterns, CI integration	pip install cloudshift, 150 patterns
MVP	Week 12 (Q3 2026)	Full pattern catalogue, CLI, Python SDK	335 patterns, 3 reference apps transformed
GA	Week 20 (Q4 2026)	IDE extensions, AI fallback, Validation Agent	IDE extension published, 500+ patterns

13. Commercial Model

13.1 Positioning

CloudShift v2 is positioned as a Smeyatsky Labs product and as a Searce delivery accelerator. It has two independent commercial tracks:

Track	Model	Target Buyer	Price Point
Open Core	Free CLI + Python SDK. Core engine and community catalogue are open-source (Apache 2.0).	Individual developers, open-source projects, GCP customer self-service	Free
Enterprise	Extended catalogue (AI/ML, finance-specific patterns), Validation Agent, audit log export, priority support.	Enterprise migration programmes, Searce client deliveries	$3,000–$15,000/migration project
Searce Accelerator Bundle	CloudShift pre-configured for Searce's GCP migration methodology. Included in migration SOW pricing.	Searce enterprise clients (AWS→GCP, Azure→GCP)	Bundled in SOW
SaaS (GA+)	Hosted API and web UI for teams that cannot self-host. Air-gapped option available.	Mid-market teams, ISVs	$500–$2,000/month

13.2 GTM Channels

- Google Cloud Partner ecosystem: CloudShift is positioned as a Searce-built Google Cloud Premier Partner accelerator, eligible for the Google Cloud Technology Partner Programme.
- Smeyatsky Labs direct: via smeyatsky.com/products/cloudshift and cloudshift.dev open-source site.
- Greyquill (Amarnath): India delivery partner. CloudShift is included in Greyquill's GCP migration service offering.
- Google PSO referral: CloudShift has been shown to Google PSO India as a refactor accelerator with expressed interest. Formal partner programme registration is the next step.
- Community: open-source release on GitHub (smeyatsky-labs/cloudshift) with docs, tutorials, and blog posts.

13.3 Revenue Projections (Conservative)

Quarter	Revenue Source	Estimate
Q3 2026 (MVP)	2x Searce client projects bundled	£20,000
Q4 2026 (GA)	4x Searce + 2x direct enterprise	£60,000
Q1 2027	Searce pipeline + SaaS launch + Greyquill	£90,000
Q2 2027	Full pipeline + Google PSO referrals	£140,000

14. Competitive Positioning

The Augment Code research identifies the 19 major refactoring tools in the market. CloudShift v2's position against the most relevant competitors is defined below.

Tool	GCP Specificity	Deterministic	Air-gapped	CLI/CI	Python SDK	CloudShift Advantage
Augment Code	None	No (AI)	No	Yes	No	GCP-specialised, compiled patterns, auditable
GitHub Copilot	None	No (AI)	No	No	No	Deterministic, CI-native, no subscription per-seat
Cline	None	No (AI)	Partial	Partial	No	Compiled rules, Python SDK, SARIF output
Aider	None	No (AI)	Partial	Yes	No	GCP-specific, no token cost per transform
Terraform Migration Tool	GCP only	Partial	Yes	Yes	No	Code + IaC in one tool, Python SDK
CloudShift v2	GCP-only (by design)	Yes (pattern engine)	Yes	Yes	Yes (Maturin)	—

(Final column N/A for CloudShift row — this is the reference row.)

🔍 The market insight from the Augment Code analysis is clear: 'Context limits will only expand as models evolve — choosing tools that already operate at repository scale protects today's refactoring investment.' CloudShift's response: compiled determinism scales infinitely. A 5-million-line monolith runs through CloudShift in minutes with zero token cost and 100% reproducible output.

15. Risks and Mitigations

Risk	Likelihood	Impact	Mitigation
SDK version drift: AWS/Azure SDKs release frequently. Patterns become outdated.	High	Medium	Pattern versioning system. CI job monitors SDK releases and flags affected patterns. Community PRs for updates.
Coverage gaps: Uncommon SDK usage patterns not in catalogue.	Medium	Medium	AI fallback (GA). Community catalogue contributions. Confidence thresholds ensure unknown patterns are flagged, not silently mis-transformed.
Tree-sitter grammar bugs on edge-case syntax.	Low	High	Error-tolerant parsing gracefully degrades. Unparseable files are flagged and skipped, not crashed. Comprehensive test corpus.
Maturin/PyO3 ABI compatibility issues on older Python versions.	Low	Medium	Strict Python version matrix in CI (3.9, 3.10, 3.11, 3.12). manylinux wheels tested on Docker before release.
Competition from Google-native migration tooling.	Medium	High	Early partner programme registration. Position as complementary to GCP tooling, not competitive. Embed in Searce SOWs before Google ships a native tool.
Build time exceeds developer patience for large repos.	Low	Medium	Parallel processing (Rayon). Incremental transformation (only changed files re-processed). Benchmark target: < 30 seconds for 100k LOC.

16. Appendix — Technology Rationale

A. Why Rust for the Core Engine

The primary alternatives considered were Python (for ecosystem familiarity) and Go (for simplicity). Rust was chosen for three reasons:

- Compiler-adjacent tooling: the tree-sitter Rust crate, syn (Rust AST library), and proc-macro tooling are all Rust-native. Working against Tree-sitter CSTs in Rust is idiomatic; doing so in Python via bindings introduces a translation overhead.
- Performance without a garbage collector: transformation of a 500k LOC repository must complete in < 30 seconds. GC pauses in Python or the JVM are unpredictable at this scale. Rust's deterministic memory management is the correct tool.
- Maturin/PyO3 ecosystem maturity: the Python extension path from Rust is well-established and production-proven (Polars, Pydantic v2, ruff). This gives CloudShift the best of both worlds: Rust performance for the engine, Python accessibility for the SDK.

B. Why Maturin (not CFFI, ctypes, or pybind11)

- Maturin produces PEP 517-compliant wheels. pip install cloudshift works without any Rust toolchain on the user's machine.
- PyO3 (Maturin's binding layer) provides safe, ergonomic type conversion between Rust and Python types with no unsafe code required.
- Maturin has first-class GitHub Actions support (PyO3/maturin-action). The release pipeline is a single workflow file.
- Alternative: CFFI requires manual C-level bindings and ctypes has no safety guarantees. pybind11 is a C++ solution and not applicable.

C. Why Tree-sitter (not ANTLR or a hand-written parser)

- Tree-sitter grammars cover 40+ languages with a single, unified Rust API. ANTLR requires per-language runtime setup and a JVM.
- Tree-sitter is incremental: re-parsing changed files in a large repo is O(changed bytes), not O(total bytes).
- Tree-sitter is error-tolerant: real-world legacy code is not perfectly syntactically valid. Tree-sitter produces a best-effort parse tree rather than failing.
- Tree-sitter S-expression queries are a clean, declarative way to express pattern matching conditions. This maps naturally to the Pattern DSL.

D. Comparison: CloudShift v1 vs v2

Dimension	v1 (Original)	v2 (This PRD)
Focus	Infrastructure-level service mapping (AWS/Azure → GCP equivalents)	Application code + IaC + CI/CD. Any code that talks to cloud services.
Engine	Rust core + Python agent layer	Rust core (cloudshift-core) + Python extension via Maturin (PyO3)
Parsing	Tree-sitter (planned)	Tree-sitter (first-class, all languages, compiled in)

Dimension	v1 (Original)	v2 (This PRD)
Pattern DSL	Not defined	TOML-based DSL with Tree-sitter S-expression queries
Python Access	Python agent calls Rust binary	Native Python extension: import cloudshift
AI Role	LLM fallback with BYOK	Identical — LLM is last resort. Compiled patterns first.
IDE Integration	VS Code + JetBrains (planned)	LSP server (cloudshift-lsp) for universal IDE support
Delivery	Web UI + CLI + IDE	CLI + Python SDK + CI/CD first. Web UI as GA add-on.

CloudShift v2 is not a rewrite of v1 for its own sake. It is a sharpening of the product to a laser-focused, developer-first, CI-native tool. The complexity of v1 (air-gap architecture, multi-surface delivery, self-learning store) is preserved but deferred to GA. MVP is a single, excellent thing: point it at a codebase, get GCP-ready code out.